

Healthcare Resource Availability: Analyzing Global Disparities

ADAN8880 - Directed Practicum in Analytics

Professor: Nurtekin Savas

Date: March 14, 2025

Group Members: Jen Jiang, Yiwen Ma

Week 7 Written Report

XGBoost for Life Expectancy Prediction

The model approach we chose this week is XGBoost Regressor. This choice makes sense for our prediction task because XGBoost is a powerful gradient boosting algorithm that efficiently handles structured data, missing values, and complex relationships between features. It is known for its high predictive accuracy and ability to prevent overfitting through built-in regularization techniques.

In this case, we aim to predict life expectancy at birth using various socioeconomic and health indicators. Since this is a regression problem with potentially nonlinear relationships, XGBoost's ability to capture intricate patterns and interactions between variables makes it an ideal choice. Additionally, its scalability allows us to process large datasets efficiently, making it well-suited for real-world prediction tasks.

Complexity of the modeling approach

The XGBoost Regressor approach introduces a moderate to high level of complexity due to its ensemble-based structure and hyperparameter tuning requirements. Unlike simple linear regression models, XGBoost builds multiple decision trees sequentially, using boosting to correct errors from previous iterations. This iterative process enhances predictive accuracy but also increases computational complexity.

Key Factors Contributing to Complexity:

1. Feature Engineering & Preprocessing:
 - a. XGBoost can handle missing values automatically, but feature selection and transformation still play a crucial role in improving model performance.
 - b. Encoding categorical variables, scaling numerical values, and handling multicollinearity add to the preprocessing effort.
2. Hyperparameter Optimization:

- a. XGBoost requires careful tuning of parameters like learning rate (eta), max depth, number of estimators, and regularization terms.
 - b. Grid search or Bayesian optimization is often necessary to find the best settings, increasing computational cost.
- 3. Computational Load:
 - a. Training an XGBoost model can be resource-intensive, especially with large datasets and deep trees.
 - b. While XGBoost is optimized for speed through parallelization, it still demands significant memory and processing power.
- 4. Interpreting Model Outputs:
 - a. Unlike simpler models, XGBoost lacks direct interpretability, making it challenging to explain individual predictions.
 - b. SHAP (SHapley Additive Explanations) or feature importance plots are often used to interpret the model's decisions.

Despite these complexities, XGBoost's ability to generalize well across diverse datasets makes it a strong choice for predictive modeling tasks where high accuracy is required.

Hyperparameters Evaluated and Rationale

To optimize the performance of our XGBoost Regressor (XGBRegressor), we evaluated several key hyperparameters to balance accuracy, generalization, and computational efficiency. Below are the hyperparameters we focused on and why they were chosen:

- 1. Learning rate (eta):
 - a. Why? Controls the step size during weight updates.
 - b. Impact: A lower value (e.g., 0.01 or 0.1) leads to slower, more stable training, reducing overfitting. A higher value (e.g., 0.3 or 0.5) speeds up training but may cause divergence.
 - c. Tuning goal: Find a balance where the model learns efficiently without overfitting.
- 2. Maximum depth (max_depth):
 - a. Why? Determines the depth of each decision tree in the ensemble.
 - b. Impact: Deeper trees capture more complex patterns but risk overfitting. Shallower trees improve generalization but may underfit.
 - c. Tuning goal: Optimize depth to prevent overfitting while maintaining predictive power.
- 3. Number of estimators (n_estimators)

- a. Why? Defines the total number of boosting iterations.
 - b. Impact: More trees improve learning but increase training time and potential overfitting.
 - c. Tuning goal: Identify the best number of trees to balance accuracy and efficiency.
4. Regularization parameters (lambda, alpha, gamma):
- a. Why? Control model complexity to prevent overfitting.
 - b. Impact: Higher values apply stronger regularization, reducing model variance.
 - c. Tuning goal: Adjust to improve generalization without sacrificing too much flexibility.

By systematically tuning these hyperparameters, we aimed to create a model that delivers high accuracy while avoiding overfitting and excessive computation.

Model Performance Metrics and Their Relevance

To evaluate the performance of our XGBoost Regressor (XGBRegressor), we used the following metrics, which align with our regression-based prediction task.

1. Mean Squared Error (MSE):
 - Measures the average squared differences between actual and predicted values.
 - Lower values indicate better model performance.
 - Makes sense because it penalizes larger errors more heavily, ensuring the model prioritizes accurate predictions.
2. Root Mean Squared Error (RMSE):
 - The square root of MSE, making it more interpretable in the same unit as the target variable.
 - Helps understand the magnitude of prediction errors in practical terms.
 - Suitable for our task since it emphasizes larger errors, which can be critical in predicting life expectancy.
3. R-squared (R^2) Score:
 - Represents the proportion of variance explained by the model.
 - Values range from 0 to 1, where a higher value indicates a better fit.
 - Useful because it helps assess how well our features explain the variability in life expectancy predictions.

These metrics were chosen because they provide a balance between measuring absolute errors (MSE, RMSE) and explaining variance (R^2), helping us optimize the model's predictive power while minimizing errors.

Calculations for all 3 variations (training and validation)

XGB default with early stopping – Train RMSE: 0.0695, Validation RMSE: 0.0840

XGB feature subsampling – Train RMSE: 0.4378, Validation RMSE: 0.3015

XGB regularized – Train RMSE: 3.0677, Validation RMSE: 2.5357

	Model	Train RMSE	Validation RMSE	Train R ²	Validation R ²
0	XGB default with early stopping	0.069544	0.083957	0.999944	0.999895
1	XGB feature subsampling	0.437757	0.301470	0.997800	0.998648
2	XGB regularized	3.067678	2.535737	0.891966	0.904340

1. XGB Default with Early Stopping:

- Training RMSE: 0.0695
- Validation RMSE: 0.0840
- Training R²: 0.99994
- Validation R²: 0.99989
- Analysis: This model achieves the lowest RMSE and the highest R² for both training and validation sets, indicating an almost perfect fit. The small gap between training and validation RMSE suggests minimal overfitting, and the validation R² close to 1 signifies that the model explains nearly all the variance in the data.

2. XGB Feature Subsampling:

- Training RMSE: 0.4378
- Validation RMSE: 0.3015
- Training R²: 0.9978
- Validation R²: 0.9986
- Analysis: This model performs slightly worse than the default version, as reflected by the increased RMSE and slightly lower R² values. The higher validation RMSE (0.3015) suggests that feature subsampling introduces some level of bias, but the gap between training and validation RMSE is wider than in the default model, indicating improved generalization at the cost of slightly reduced accuracy.

3. XGB Regularized:

- Training RMSE: 3.0677

- Validation RMSE: 2.5357
- Training R^2 : 0.8919
- Validation R^2 : 0.9043
- Analysis: This model demonstrates the weakest performance, with significantly higher RMSE values and much lower R^2 scores. The drastic increase in RMSE compared to the other two models suggests that strong regularization has led to underfitting. Both the training and validation R^2 values are much lower than in the other models, showing that the model is unable to capture key patterns in the data.

Comparison of Validation Metrics Across Variations:

- The validation RMSE increases significantly as regularization is applied, moving from 0.0840 in the default model to 0.3015 with feature subsampling, and finally reaching 2.5357 with heavy regularization.
- The validation R^2 remains high for the first two models (0.99989 and 0.9986, respectively), indicating that both models explain nearly all the variance in the validation set. However, in the regularized model, the validation R^2 drops significantly to 0.9043, confirming that the model struggles to explain the variance in the data.
- The default model with early stopping maintains the best balance, as its validation RMSE is the lowest and its validation R^2 is the highest, suggesting superior predictive power and generalization.
- The feature subsampling model, while slightly less accurate, may be preferable if generalization is a concern, as it introduces some regularization while still maintaining high predictive accuracy.
- The regularized model is not ideal in this scenario, as excessive regularization has drastically reduced performance, leading to significant underfitting.

Key Insights:

- The default model with early stopping offers the best trade-off between accuracy and generalization.
- Feature subsampling introduces a small trade-off in accuracy but might help in cases where reducing overfitting is necessary.
- Excessive regularization negatively impacts model performance, leading to underfitting and poor generalization.

- Choosing the right model variation depends on the specific use case; however, in this case, the default model with early stopping is the clear winner in terms of both training and validation performance.

Table of our variation

Model Variation	Training RMSE	Validation RMSE	Training R ²	Validation R ²
XGB Default with Early Stopping	0.0695	0.084	0.99994	0.99989
XGB Feature Subsampling	0.4378	0.3015	0.9978	0.9986
XGB Regularized	3.0677	2.5357	0.8919	0.9043

Best Model for the Week: XGB Default with Early Stopping

Among the three XGBoost variations tested, the XGB default with early stopping is the best-performing model based on multiple evaluation metrics. The reasons for selecting this model are:

1. Lowest RMSE (Best Predictive Accuracy)**
 - Training RMSE: 0.0695
 - Validation RMSE: 0.0840
 - This model has the lowest root mean squared error (RMSE) on both the training and validation datasets, indicating that its predictions are the closest to the actual values.
2. Highest R² (Best Variance Explanation)
 - Training R²: 0.99994
 - Validation R²: 0.99989
 - The R² values are extremely high, showing that the model explains nearly all the variance in the data. A higher R² means better predictive performance, and this model outperforms the others in this regard.
3. Balanced Generalization (Minimal Overfitting)
 - The gap between the training and validation RMSE is minimal (0.0145), indicating that the model generalizes well to unseen data.

- While the XGB feature subsampling model also generalizes well, it has a significantly higher RMSE (0.3015 for validation), making it less accurate overall.

4. Avoids Underfitting Issues

- The XGB regularized model performs poorly due to excessive regularization, with a validation RMSE of 2.5357, making it a clear underperformer. The drastic drop in R^2 to 0.9043 suggests that the model fails to capture key patterns in the data.

The XGB default with early stopping is the best model because it achieves the best trade-off between accuracy and generalization. It minimizes error while ensuring that it does not overfit or underfit the data. This makes it the most reliable choice for deployment or further fine-tuning.

Test Set

Final Model – Test Set – Train RMSE: 3.0677, Validation RMSE: 2.6919

Final Model performance on test data:
 Train RMSE: 3.0677, Validation RMSE: 2.6919
 Train R^2 : 0.8920, Validation R^2 : 0.8916

Test set results:

Model	Train RMSE	Validation RMSE	Train R^2	Validation R^2
Final Model – Test Set	3.067678	2.691949	0.891966	0.891576

We conducted further testing on our test data, and the results confirmed that the model's performance remains consistent with the validation data. The test RMSE of 2.6919 and test R^2 of 0.8916 closely align with the validation RMSE of 2.6919 and validation R^2 of 0.8916, reinforcing that our model generalizes well to unseen data. This consistency suggests that the validation set provided a reliable estimate of real-world performance, further validating the model's robustness and stability.